

Helix Cheatsheet

Helix Cheatsheet: Normal mode, Insert mode, LSP, TS, pickers, SPC ? (black-hole), Kakoune mode

HELIX MINDSET (start here)	
:tutor	interactive built-in tutorial — run it!
SPC ?	command palette / key discovery (use it constantly)
selection first	select target(s) before the verb (d/c/y etc)
multi-sel core	C/Alt-C add cursors; s select matches inside
x	select line (repeat x to extend downward)
think ranges	cursor = 1-char sel; act on any sel(s)
no visual mode	v = select/extend mode (motions now extend)
avoid dw etc	don't port Vim habits — select then act
<i>First thing: run :tutor. Use SPC ? constantly to discover.</i>	
MODES	
Normal	default — navigation + commands
Insert	i / a / o / I / A / O — minimal bindings
Select (v)	movements <i>extend</i> selections instead of moving cursor
goto (g)	g prefix — jumps, LSP goto, file ends
match (m)	m prefix — brackets, surround, textobjects
view (z / Z)	z = temp view ops; Z = sticky view (Esc to exit)
window (C-w)	split / nav / swap windows (Vim-like)
space (SPC)	pickers + LSP + clipboard (leader-like)
:	command / ex mode (typable commands)
<i>Esc always returns to Normal. Changes only check-point on Esc from Insert.</i>	
MOVEMENT (normal mode)	

h / j / k / l	left / down / up / right (visual lines for j/k)
w / b / e	next/prev word start, word end
W / B / E	WORD (non-word chars separate)
t{c} / f{c}	till / find char (works <i>across lines</i>)
T / F	prev till / prev find
Alt-.	repeat last f/t/m/[motion
Home / End	line start / end
O / \$	start / end (goto mode has more)
C-u / C-d	half page up/down (cursor moves with view)
C-b / C-f	page up / down
C-o / C-i	jump list backward / forward
C-s	save current selection to jumplist
gg / G	file start / end (also via g mode)
<i>f/t/F/T are not line-limited like Vim. Use Alt-. to repeat.</i>	
GOTO MODE (g prefix)	
gg	file start (or nG for line n)
ge	file end
gh / gl	line home / end
gs	first non-blank
gt / gc / gb	view top / center / bottom
gd	goto definition (LSP)
gy	goto type def (LSP)
gr	goto references (LSP)
gi	goto implementation (LSP)
gf	open file under cursor / sel
ga	last accessed file
gm	last modified file
gn / gp	next / prev buffer
g.	last modification in file
gw	word jump labels (type chars)
gj / gk	move by <i>textual</i> line (not wrapped)
SELECTION MANIPULATION	

x	select current line (repeat = extend down)
X	extend selection to full line bounds
Alt-x	shrink to line bounds
%	select entire file
s	select <i>all regex matches</i> inside current sel(s)
S	split selection(s) on regex
Alt-s	split selection on newlines
Alt-	merge all selections
Alt-_	merge consecutive selections
&	align selections into columns
-	trim whitespace from selections
;	collapse selection to single cursor (keep pos)
Alt-;	flip anchor ↔ cursor
Alt-:	force selection forward
,	keep only primary selection
Alt-,	remove primary selection
C	duplicate selection + cursor to next line
Alt-C	duplicate selection + cursor to prev line
(/)	rotate main selection backward / forward
Alt-(/Alt-)	rotate sel contents (parens)
<i>This is the heart of Helix. Master x, s, C, Alt-C, ; and % early.</i>	
TS SYNTAX SELECTIONS (Alt)	
Alt-o / Alt-Up	expand to parent syntax node
Alt-i / Alt-Down	shrink to child syntax node
Alt-p / Alt-Left	previous sibling node
Alt-n / Alt-Right	next sibling node
Alt-a	select all siblings
Alt-I	select all children of node
Alt-e	move to end of parent node
Alt-b	move to start of parent node
<i>Requires tree-sitter grammar. Combine with ma/mi for powerful selects.</i>	
CHANGES (act on selections)	

d	delete selection (yanks to ")
Alt-d	delete, <i>no yank</i> (black-hole)
c	change = delete + insert
Alt-c	change, no yank
y	yank selection(s)
p / P	paste after / before selection
"r y / "r p	use register r for yank/paste
r{c}	replace each char in sel with c
R	replace selection(s) with last yank
~	toggle case of selection
`	lowercase selection
Alt``	uppercase selection
> / <	indent / dedent selection
=	format selection (LSP formatter)
C-a / C-x	increment / decrement number under cursor
J / Alt-J	join lines (no space / with space)
<i>All operations apply to every active selection simultaneously.</i>	
INSERT	
i	insert before sel
a	append after sel
I	insert at line start
A	insert at line end
o	open below + insert
O	open above + insert
.	repeat last insert
Esc	back to normal (checkpoint)
C-s	commit undo checkpoint
C-x	completion menu
C-r	insert register
C-w/Alt-BS	del word back
Alt-d/Alt-Del	del word fwd
C-u	kill to line start
C-k	kill to end
C-h/BS	del char back
C-d/Del	del char fwd
C-j/Enter	newline
<i>Minimal insert by design. Disable arrows in config later.</i>	
SELECT / EXTEND MODE (v)	
v	enter — all movement now <i>extends</i> sel
(normal keys)	h/j/k/l/w/... extend instead of move
g...	goto motions also extend
n / N	in select: add next/prev search match
Esc	back to normal (keep extended sel)
<i>Toggle v on/off while searching to accumulate</i>	

<i>matches into selections.</i>	
SEARCH	
/	search forward (regex, stores in / reg)
?	search backward
n / N	select next / prev match (replaces sel)
*	search using current sel (word boundaries)
Alt-*	search using current sel (exact)
"r /	use register r for search pattern
<i>Combine with v (select mode) or s to accumulate many matches.</i>	
MATCH MODE (m prefix)	
mm	goto matching bracket (TS)
ms{c}	surround add (e.g. ms")
mr{old}{new}	replace surrounding (mr"")
md{c}	delete surrounding (md")
ma{obj}	select around textobject
mi{obj}	select inside textobject
<i>ma/mi for around/inside. Counts work (2ms"). Requires TS for f/t etc.</i>	
TEXTOBJECTS & SURROUND	
w/W	word/WORD
p	paragraph
f	function (TS)
t	type/class (TS)
a	arg/param (TS)
c	comment (TS)
T	test (TS)
g	change/hunk
m	closest pair
([' "	specific pair
ms{c}	add surround
mr old new	replace surround
md{c}	delete surround
<i>ma f = around fn. % then s then mr turns many.</i>	
UNIMPAIRED NAV ([J)	
ld / [d	next / prev diagnostic (LSP)
ld / [D	last / first diagnostic
lf / [f	next / prev function (TS)
lt / [t	next / prev type/class
la / [a	next / prev arg/param
lc / [c	next / prev comment
lT / [T	next / prev test
lp / [p	next / prev paragraph
lg / [g	next / prev change
lG / [G	last / first change
]Space / [Space	add blank line below / above
SPACE MODE (SPC prefix)	

f / F	file picker (workspace / cwd)
b	buffer picker
j	jumplist picker
g	changed files (git) picker
k	hover / docs popup (LSP)
s / S	document / workspace symbols (LSP)
d / D	document / workspace diagnostics
r	rename symbol (LSP)
a	code action (LSP)
h	select references to symbol
,	re-open last fuzzy picker
w	enter window mode
c / C	toggle line / block comments
Alt-c	toggle line comments
p / P	paste system clipboard after / before
y / Y	yank sel(s) / main sel to clipboard
R	replace selections with clipboard
/	global search (regex in workspace)
?	command palette (discover keys + cmds)
<i>SPC ? is your best friend when learning. SPC , resumes global search etc.</i>	
WINDOW MODE (C-w)	
C-w / w	next window
v / C-v	vertical split
s / C-s	horizontal split
h/j/k/l	move to left/down/up/right window
H/J/K/L	swap window left/down/up/right
q / C-q	close window
o / C-o	only current window (close others)
f / F	goto file in sel (hsplit / vsplit)
VIEW MODE (z / Z)	

z / c	center line vertically
t	align line to top
b	align line to bottom
m	align view horizontally to middle
j/k	scroll view down/up (no cursor move)
C-f / C-b	page down/up
C-u / C-d	half page (cursor moves)
<i>Z makes it sticky (Esc exits). Great for reading without moving point.</i>	
PICKER KEYS	
Tab / C-n / Down	next entry
S-Tab / C-p / Up	prev entry
C-u / PgUp	page up
C-d / PgDn	page down
Home / End	first / last
Enter	open selected
Alt-Enter	open in background (keep picker)
C-s	open in horizontal split
C-v	open in vertical split
C-t	toggle preview
Esc / C-c	close picker
PROMPT / COMMAND-LINE	
Esc / C-c	close / cancel
C-b / Left	char left
Alt-b/C-Left	word left
C-f / Right	char right
Alt-f/C-Right	word right
C-a / Home	prompt start
C-e / End	prompt end
C-w / C-BS	delete word back
Alt-d/C-Del	delete word fwd
C-u	kill to start
C-k	kill to end
C-r	insert register (then name)
C-s	insert word under cursor
C-p / Up	history prev
C-n / Down	history next
Tab / S-Tab	next / prev completion
REGISTERS	

"r before op	use register r (yank/paste/change/delete)
"	default yank register (last yank)
/	last search pattern
:	last command
@	last macro
+	system clipboard (yank/paste join)
*	primary selection clipboard
.	current selection contents (read-only)
%	current filename (read-only)
#	selection indices (1,2,...) (read-only)
-	black hole (write discards)
<i>*+y yanks to system. Multiple sels joined by newline on clipboard yank.</i>	
SHELL & MACROS	
 	pipe sel thru cmd (replace w/ stdout)
Alt- 	pipe to cmd, ignore out
!	insert cmd stdout before sel
Alt-!	append cmd stdout after
\$	keep sels where cmd==0
:sh / :!	run shell cmd
:pipe / : 	pipe sels
Q	start/stop macro rec
q	replay macro
<i>Macros still experimental — prefer multi-sel + s for most bulk edits.</i>	
TYPABLE COMMANDS (:)	

:w / :write	save
:q / :quit	close view
:q!	force quit
:wq / :x	write + quit
:bc / :bclose	close buffer
:bco	close other buffers
:bn / :bp	next / prev buffer
:format / :fmt	format via LSP or formatter
:cd	change directory
:pwd	show cwd
:reload	discard changes + reload file
:theme	switch theme (no arg = current)
:tutor	open interactive tutor
:config-open	edit your config.toml
:lsp-restart	restart language servers
:set / :toggle	runtime option (e.g. search.smart-case)
<i>Many more; use SPC ? or : then Tab to explore. :reflow for hard wrap.</i>	
LSP WORKFLOW (built-in)	
SPC k	hover docs
SPC s	document symbols picker
SPC r	rename
SPC a	code action (quickfix/refactor)
gd / gy / gr / gi	goto def/type/refs/impl
ld / ld	next/prev diagnostic
SPC d	document diagnostics picker
=	format selection / :format
inlay hints	toggle via config or command
<i>No plugin install. Just open project with hx . — servers auto-start per lang.</i>	
MULTI-SELECTION TACTICS	
C/Alt-C	add cursor below/above
% s pat Enter	select every match of regex in file
* or Alt-* then n	search sel, add more matches
v n / v N	in select mode: accumulate search hits
SPC / then pick	global search + open files
ma f then c	change inside every function at once
x C x C	select several lines then edit together

<i>This replaces 90% of what you used macros / :%s for in Vim.</i>	
CONFIG & BEHAVIORS	
theme	"onedark" (or other)
line-number	relative (in [editor])
auto-format	true (on save)
auto-save	false
cursorline	true
cursor-shape	normal=block insert=bar
file-picker	hidden=true
keys.normal	add your maps
trim-ws	on write (opt)
jumplist	C-o/C-i +C-s
undo	persistent history
EditorConfig	respected
smart-tab	smart indent
<i>File: /.config/helix/config.toml . Use :config-open + :config-reload</i>	
COMMON GOTCHAS (from nvim)	
no :%s	use select-all + c or global search + replace
no visual block I	use C to add cursors then i or c
dd is xd	or x d; line select is explicit
yyp is x y p	select line, yank, paste
f only line?	no — multiline in Helix
registers	mostly same, but +/* special for clip
plugins	almost none needed — everything built-in
QUICKSTART CHEAT	
hx file.rs	open
SPC f	find files
SPC / foo	project search
x d	delete current line
% s foo Enter c	change every foo in file
C C c bar Esc	add cursors on 3 lines, change all
ma f c	change inside all functions
SPC a	apply code action at point
SPC r newname	rename symbol
:w	save
:bc or SPC w?	close buffer
<i>SPC ? for discovery. See official keymap.html for full list.</i>	